Small Size Soccer Goalie Through DeepRL

Devin Schwab
Robotics Institute PhD
dschwab@andrew.cmu.edu

Rui Silva
CSD PhD
ruisilva@cmu.edu

Sai Prabhakar
Robotics Institute Masters
spandise@andrew.cmu.edu

1 Introduction

RoboCup, is an international competition where teams of researchers attempt to create AI agents capable of autonomously playing the game of soccer [4]. In the last two years, the CMDragons team took first and second place in the league, however, there is still room for improvement. Specifically, the CMDragons' goalie uses a simple hand-coded policy. It first tries to capture the ball by calculating an approximate interception location. If that fails, then the robot tries to throw itself in front of the ball. Given the simplicity of this policy, it is interesting to see if the goalie can be improved by leveraging recent techniques in Deep Reinforcement Learning (DeepRL).

In the last few years, DeepRL algorithms have become a major focus of research after a number of successful applications by the Google DeepMind team. In 2013, DeepMind showed that using deep neural networks, they could train an RL agent that achieves super-human performance on several Atari games [11, 12]. DeepMind later published the Deep Deterministic Policy Gradient (DDPG) algorithm, which extended DQN to support continuous action space [10]. Thus, making DeepRL algorithms far more useful for robotic applications. More recently, Deep Mind also leveraged DeepRL techniques to create the first AI agent that could beat one of the world's best Go players [14]. Given these successes, it is likely that these techniques can be useful for creating parts of a RoboCup agent.

In fact, some work demonstrating the viability of DeepRL algorithms in the area of robot soccer has been published. Hausknecht and Stone have extended the DDPG algorithm to support parameterized action spaces [6, 7], and tested this new algorithm using the Small Size RoboCup domain. Parameterized action spaces are domains that have multiple discrete actions and each discrete action has one or more continuous parameters associated with it. In the robot soccer domain these correspond to actions like "Kick(power)", where "Kick" is a discrete action and "power" is a continuous parameter associated with the action kick. Using this algorithm, they train a single agent to navigate to the ball and kick it into an undefended goal.

Normalized Advantage Functions (NAF) [5] have been proposed as an alternative to the actor-critic based DDPG algorithm. This technique uses locally linear model approximations to accelerate the learning. The experiments show that for large networks, NAF performs better over a large range of domains. However, this approach has not been extended to parameterized action space, which has been demonstrated as a useful representation for RoboCup soccer tasks [7].

In the Parameterized Action space paper, reward shaping is used due to the sparsity of the reward function in robot soccer. However, there has been some recent research in biasing the replay memory to ensure that important samples are used more often [13]. By biasing the replay memory using a distribution based on the temporal difference (TD) error, it has been shown that an agent can learn to play Atari games with far fewer updates and samples. This technique may be useful as an alternative to reward shaping in robot soccer domains.

10807 Class Project (2016).

2 Methods

In this project we attempt to learn how to defend a goal in the SSL RoboCup league. The original proposal split the project into two phases: first just blocking the ball from going into the goal and secondly gaining control of the ball so that the goalie can pass it to a teammate. However, even just simple blocking of the goal turned out to be a difficult learning task. So instead our project focuses on just learning to block the goal. In an attempt to understand why this task is difficult we also train an agent for many different simpler tasks and explore when the agent starts having difficulty learning.

2.1 Algorithms

We utilized two existing DeepRL algorithms: DQN and DDPG, which we implemented from scratch. To verify the correctness of our algorithms we tested them on environments from the OpenAI gym. Both of our algorithms are able to converge to reasonable policies on these tasks. We used Tensorflow [1] and Keras [2] for the implementation. Using Tensorflow and Keras meant we could skip the difficulties of implementing good optimizers (e.g. ADAM [9]), backprop, etc. and instead focus on the actual RL algorithms.

The Deep Q-Network (DQN) algorithm [11, 12] works for discrete action Markov Decision Processes (MDPs). The algorithm was originally introduced in the context of learning to play Atari games. The idea is to use a deep feed forward neural network to approximate the Q-function of an MDP. Each neuron in the output layer corresponds to the Q-value of a specific action in the MDP. So computing the Q-values of all actions for a state only requires a single pass. To find the optimal action just take the neuron with the best Q-value for the state. The DQN paper introduced the idea of a target network and replay memory in order to improve training stability. This algorithm was tested with a discretized action version of the goal blocking task.

The Deep Deterministic Policy Gradient (DDPG) algorithm is an extension of the DQN algorithm for continuous action spaces. This is an actor critic based method. Instead of one single network that outputs the Q-values for every action, there are now two networks: an actor and a critic network. The actor network is responsible for mapping a state to a continuous action. The critic network is responsible for evaluating the state and the actor output. Gradients flow from the loss of the critic back through the network so that the estimate of an action's Q-value improves at the same time that the agent's actions improve. During execution of the policy only the actor network is consulted. In theory a policy in continuous action space can achieve better outcomes because it has a higher control resolution.

2.1.1 Inverted Pendulum

To test our implementations of DDPG and DQN we used different OpenAI gym environments. In this section we will focus on the DDPG algorithm applied to the inverted pendulum domain. In the inverted pendulum task, the agent applies torques to a single link pendulum in order to balance it in the upright position. The agent receives negative rewards proportional to the distance between its current angle and the upright angle. The actions an agent outputs are real valued and correspond to the torque in N/m. An optimal policy should swing the pendulum back and forth until it builds up momentum to reach the top. Once upright the policy needs to continue balancing the pendulum.

The pendulum task is a good test task because it can be learned with few samples, and yet the final policy varies a lot according to the state. The torque values required change based on the angle due to gravity. The policy must also switch from building up momentum to removing momentum in order to keep the pendulum balanced. This environment also gave us a chance to explore how policies trained with DDPG perform during training and how the hyperparameters affect training.

2.2 Soccer Simulator

While open source SSL simulators, such as the Half Field Offense (HFO) simulator, exist [8], we have chosen to train our agent using the custom CMDragons' simulator, refer Figure 8. While the HFO simulator has a nice API designed for learning it uses parameterized actions which aren't compatible with the base DDPG and DQN algorithms we explore in this project.

The CMDragons' simulator is also designed to work with the CMDragons' code. This means that in future work we can evaluate how our goalie performs against the CMDragons' offense. The CMDragons' have won first and second in the last two competitions, so this would be a very realistic test.

Long term, any trained agent would need to work on the real robots. The CMDragons' simulator has a very good model of the real robots used by the team. Therefore, any agent trained in this simulator is far more likely to work on the real robot. Not only do the physical constraints of the simulation match the robot, but things like latency and timing constraints are simulated.

Finally, the CMDragons' simulator has an "autoref" [15] module, which can automatically referee games. In the future we would like to evaluate goalie agents during actual game play. Using the autoref module we could run thousands of full games in the simulator in order to collect performance statistics such as number of shots blocked.

While the simulator already provided an accurate simulation, it did not provide Python APIs. In order to use Tensorflow [1] and Keras [2] we had to spend significant time reverse engineering the network communication protocol and writing Python bindings. We developed a Python interface to the simulator that can spawn robots and balls, send velocity commands to the robot, and retrieve the positions and velocities of objects in the field.

The simulator also used to run asynchronously from the agent giving commands. If the real robots do not receive commands at 60Hz, then missed updates will send a "do nothing" robot command. The simulator behaves the same way. During learning updates the agent can take longer than 1/60 second to update all its weights, meaning the state can drastically change before it tries to take its next action. To fix this, we extended the network communication protocol so that when learning, the simulator waits for commands from the agent before simulating another physics update.

The simulator provides position and orientation (r_x, r_y, r_θ) for each robot on the field along with the translational and angular velocities $(v_{rx}, v_{ry}, v_{r\theta})$. It also provides the position and velocity of the ball $(b_x, b_y, v_{bx}, v_{by})$. The physics time step is 60 Hz, which matches the real robot sensor update rate. In the real SSL games, there is a central vision server which provides this same information.

The appendix shows an image of the simulator running the full goalie training environment.

2.3 Soccer Simulator Training Environments

Ideally we would be able to train the goalie by just playing games of soccer. In theory, this would give the agent the best statistics about the real soccer games and therefore it would be able to find the best policy. However, this is not possible in practice due to sparsity of the reward signal [1] and the large state space induced by all of the robots on the field.

Each of the simulations using the CMDragons' simulator contains one goalie robot and one ball. The difference between the scenarios is the spawning locations and velocities of the robot and ball as well as the different constraints on the actions and states. The base environment is a 10 element state vector containing: $(r_x, r_y, r_\theta, v_{rx}, v_{ry}, v_{r\theta}, b_x, b_y, v_{bx}, v_{by})$. The agent can send $v_{rx}, v_{ry}, v_{r\theta}$ commands at 60 Hz simulation time. The commands are in mm/s and rads/s. For this project we restrict the goalie to staying on the goal line. So the agent only controls v_{ry}. We also use a preprocessor to take a subset of the full state space, specifically: $(r_y, v_{ry}, b_x, b_y, v_{bx}, v_{by})$.

The default command rate of 60Hz is also fairly high, so we add a "frame skip" hyperparameter. The agent will repeat the command for the number of frames specified in this parameter. Intermediate frames are ignored by the agent.

We would like to use a reward function that gives -5 for scored goals, -10 for leaving the field, +1 for first contact with the ball and +5 for blocking the goal. However, as we found out, this reward is far too sparse even with the frame skipping set to a high value. Essentially, most samples have zero reward, so the batches from the experience replay end up not having much information to learn from.

To work around this issue, we added reward shaping to the environment. We want the agent to learn to get to the intersection point of the ball and the goal line before the ball reaches that point. So

[1] Shots on goal do not occur often due to defense

we add an additional negative reward that is proportional to the distance between the robot and the ball-goal line intersection point. With this reward the agent can actually learn reasonable policies.

Learning to block goals coming from any direction with reasonable velocities and random goalie starting locations is a hard problem. Not only is the state space large due to the number of dimensions and the field size, the action space is continuous. The dynamics of the environment are also much more complex than previous work in the Atari games. Not only does the robot have momentum, it has acceleration constraints and different maximum velocities depending on the direction the robot is traveling with respect to its "front" direction. The ball does not travel with constant velocity, instead it slows down due to rolling friction.

Given these complexities, we broke the problem down into a number of simpler problems of increasing complexity. In each environment we explore how well the agent performs and what parameters work well. The next sections describe each of these environments.

2.3.1 Discrete Action Soccer

In this environment the goalie spawns randomly on the goal line and the ball spawns in a random field location with a random velocity vector to the goal. The action space is discretized to three actions: max v_{ry}, min v_{ry}, and zero velocity. The reward function is the same as the one previously described.

This environment was designed to test whether the agent can learn the correct direction it should be moving in different states. This is slightly simpler than the continuous action space because the agent only needs to learn direction of movement, not the magnitude of the movement. This is similar to games such as Atari breakout or pong, where the agent must learn to intercept a ball using a paddle that moves along a line. However, unlike the Atari games the goalie robot has to deal with momentum, acceleration constraints and the ball changing velocity due to friction.

2.3.2 Fixed Starting Ball Velocity

In this environment the goalie spawns at one goal post while the ball spawns a fixed position with a fixed velocity towards the goal. Barring some stochasticity in the simulator, the trajectory the agent follows should remain the same between all episodes. Unlike the previous environment the actions are continuous. So now the agent's policy must learn not only the proper sign of the velocity command, but the proper magnitude.

This is more complex than the previous scenario because of the continuous action space, but still far simpler than the full goalie task as the ball and goalie positions always start the same. The point of this experiment was to verify that DDPG could learn at least one ball blocking trajectory. This allowed us to test our network capacity so that we had a lower bound on the size of the network needed to learn the full goalie task. It also gave us a sense of the hyperparameter values needed.

2.3.3 Variable Ball Velocity

This environment is almost the same as the fixed ball velocity environment, but now the ball can spawn with different velocities. This means that the states encountered by the agent will vary more than in the previous case. However, the expected policy should remain the same. As it is in the agent's best interest to reach the interception point as soon as possible due to the reward shaping. The point of this environment was mainly to test how the agent responds to more variability in the state space.

2.3.4 Variable Ball Directions

In this environment, the goalie spawns at a random location on the goal line and can only move up and down the line. Balls can spawn at any position in the half field with variable velocities towards the goal. The rewards remain the same as in the other soccer environments. This is the most difficult to learn task in the project. This is also equivalent to the environment proposed in the initial project proposal.

3 Results

In this section we will describe the hypotheses and results from each of the test environments described in the previous section. Details about hyperparameters, network architecture, layer sizes, etc. will also be presented.

3.1 Inverted Pendulum Experiments

We trained the inverted pendulum agent with the DDPG algorithm. The actor network had three hidden fully connected layers using ReLU activations. Each layer had 16 units. The actor network output was a fully connected layer with one unit and a linear activation. The critic network had three hidden, fully connected layers using ReLU activations. Each layer had 32 units. The critic output was a fully connected layer with one unit and a linear activation.

A replay memory of size 10,000 was used. Prior to training 10,000 random samples were collected using an Ornstein-Uhlenbeck process. This process is essentially a Gaussian distribution with bias towards the previously output value. The soft target network update rate was 1×10^{-3}. Training used a batch size of 32 and the RMSProp optimizer with a base learning rate of .001. Gradients were clipped between -10 and 10.

Based on the results in the DDPG paper for this same task, we expected this agent to converge to a reasonable policy in under 50,000 updates. We logged the rewards seen in each batch of samples as well as the target-q values. We expected the average rewards in each batch to increase over time as the replay memory filled up with samples from better and better policies. We also expected to see the average target q values to stabilize and in the long run improve. The target q values need to converge in order for the Q-network to converge to its final value. Furthermore, we expect the value to go up in the long run because as the policy gets better, the samples get better, and therefore the predicted q values for the batches should improve.

Figure 1 shows the experimental results for the inverted pendulum task. The left plot shows the minimum, maximum, and average rewards for the pendulum task vs number of updates performed. As expected the rewards improve overtime and stabilize under 50,000 updates. The middle and right plot shows the average batch rewards and batch target q values as training progresses. The average rewards seen during a training batch increase over time as expected. The target q values stabilize at around 10,000 updates which is right about when the policy starts improving. As the policy improves samples with better q-values fill up the replay memory so the target q values improve over time.

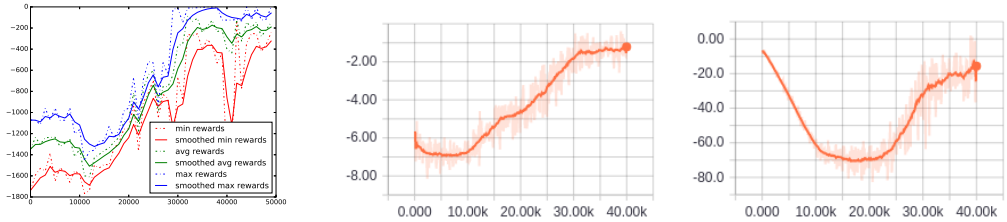


Figure 1: Inverted pendulum results. (Left) Learning curve. Total received reward averaged for five different runs vs number of training updates. (Middle) Average rewards seen during each training batch vs number of updates. (Right) Average target Q values in training batch vs number of training updates to policy.

3.2 Discrete Action Soccer

We discretized the velocity input to the simulator to max v_{ry}, zero, and min v_{yy}. This was mapped to neurons $0, 1, 2$ respectively. We trained the agent using our DQN implementation using network of four hidden layer of sizes 16, 100, 100, 100, which was chosen by our intuition. The hidden layers used ReLU activation and had batch normalization before them.

A replay memory of 20,000 was used. Prior to training 20,000 random samples were collected using uniform randomly sampled actions. The training was done using a batch size of 64 using Adam optimizer.

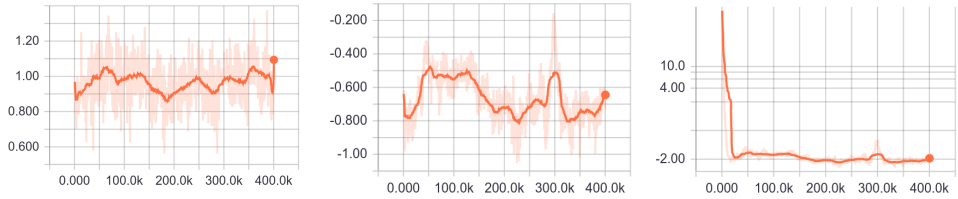


Figure 2: Discrete Action Env: (Left) Mean action, (Center) Mean reward, and (Right) Mean target-q values obtained during training.

Figure 2 shows the mean action, reward and target-q values received for the task vs number of updates performed. Since the task is an approximation of the original, we expect the network to mimic the policy fairly accurately. We expect the policy to converges and to be able block at least few of the goals. From the Figure 2, we find that the network did converge quickly based on the target-q values. We also note that the mean actions outputted by the network is around 1, which corresponds to zero velocity. This is expected as the ball position and robot position are initialized at random for each episode. The policy learned during the highest reward phase around 60,000 updates the goalie is able to head toward the ball direction but is unable to stop the ball, which again confirms that the original policy is very complex.

3.3 Fixed Ball Starting Velocity

We trained an agent using the DDPG algorithm. The actor network used batch normalization layers before each hidden layer. The actor network contained one hidden, fully connected layer with 30 units using ReLU activation. The actor output network was a fully connected layer with 1 output unit and a tanh activation. When applying actions from the actor network the output of the tanh function was multiplied by the minimum and maximum values of the velocity command.

The critic network also used batch normalization before each hidden layer. The state variables went through a fully connected hidden layer with 30 hidden units and ReLU activation before being merged with the output of the actor layer. The merged layer then went through another fully connected layer with 30 units and a ReLU activation. The output layer was fully connected layer with one output and a linear activation. The appendix shows a graphical view of this network.

A replay memory of 10,000 was used. Prior to training 10,000 random samples were collected using an Ornstein-Uhlenbeck process. We chose the noise parameters by tweaking the mean, standard deviation and persistence bias until the agent was observed being able to both hit maximum velocities as well as change direction during an episode. This time the Adam optimizer was used with a batch size of 512. The parameters were chosen based on the results of the previous experiments along with some random exploration of parameters for this particular environment.

Figure 3 show the minimum, maximum and average rewards for this task vs the number of updates performed. Because the agent has to learn only one trajectory, it was expected that the agent would converge to a policy that allows it to perfectly block all shots.

Unfortunately, we see that around 40,000 updates the agent converges to a policy that can block all of the test shots. However, within 20,000 the agent unlearns this policy. We tried running this experiment for longer, about 200,000 updates, but the agent would just oscillate between a good policy and the bad policy. Despite playing with hyperparameters, we were not able to get the network to stabilize. When we retrained the network with a larger replay memory we got similar results. Our current t is that our target update ra[illegible]n lead to instability in the training.

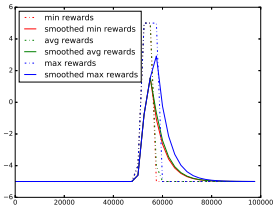


Figure 3: Fixed ball evaluation results.

To evaluate things better we looked at the performance of the best agent seen during training (i.e. the network at around 40,000 update steps). We tested the policy five times just to account for simulator errors and found that it blocked the shots with 100% accuracy.

3.4 Variable Ball Velocity

We trained the same size network as the fixed ball starting velocity environment. The same hyperparameters were used to train this network.

Figure 4 show the minimum, maximum and average rewards for this task vs the number of updates performed. Because there is more stochasticity in this environment the evaluations used 20 different episodes for each evaluation point. We see that the maximum reward increases over time, as does the average. The percentage of shots blocked also increases towards the end of the training, with a maximum of 80% of the shots blocked.

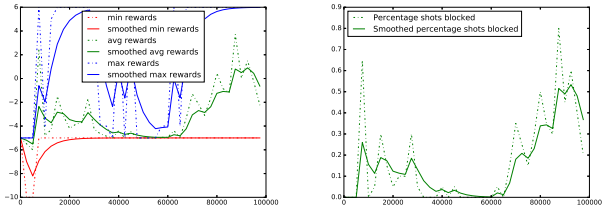


Figure 4: Variable Ball Velocity environment. (Left) Maximum, Minimum, and Average total reward received over 20 test episodes vs number of training updates performed. (Right) Percentage of shots blocked vs training updates.

3.5 Variable Ball Directions

This environment corresponds to the full task of blocking arbitrary balls while remaining on the goal line. Initially we tried training in this environment with the sparse reward function, but as mentioned the agent failed to learn due to the sparsity. Figure 6 shows the results of training with the unshaped rewards. The minimum rewards received improves overtime, but the average and max don't improve much. We had originally planned to try out the prioritized replay memory, but settled on doing the more traditional reward shaping when even simpler environments did not immediately learn perfect policies.

Figure 6 show the results for this experiment. The agent improves over time, but never achieves perfection. In many tests the DDPG policy appears to move slower than the speed needed to intercept the ball. We expected the agent to learn a "bang-bang" controller to get in front of the ball. That is, most actions should be full speed or no speed. Figure 5 shows the distribution over actions from the actor network as training progressed. The backmost distribution is from the beginning of training, and the front most distribution is from the end of training. We see that in the beginning the agent makes mostly small actions. As the training progresses we begin to see peaks at the extremes of the action space.

Our hypothesis was that the network capacity was too small to learn a proper controller. So we ran two more tests with the same hyperparameters and same architecture but more hidden units in each layer. We tried 60 hidden units in each layer and 100 hidden units in each layer. The results are very similar to the 30 hidden unit network, so we have omitted the results from the report.

4 Discussion

In this project, we have successfully implemented two of the most widely known DeepRL algorithms: DQN and DDPG. In addition to testing them on a number of OpenAI gym environments, we applied these algorithms to the problem of learning a goalie policy in Small Size League RoboCup. In addition to implementing the RL algorithms, we put a significant amount of effort into improving the CMDragons' simulator so that we had a good training environment for this project and future work.

At first glance defending a ball from entering a goal seems like a solved problem due to similarities in Atari games such as breakout and pong. However, as we show in this project, the additional

complications of increased state space, continuous actions, and complex physical dynamics makes this a much harder problem for a reinforcement learning agent.

To help understand these complexities we implemented a number of simplified domains and trained agents on each. We analyzed the performances to see what changes affected the agent's learning and how the agent's final policy was affected. We found with discrete actions that the agent is capable of learning the proper direction it should be moving, but had a harder time learning the magnitude of the velocity in the continuous action case. In the environment where the ball varies in starting velocity the agent improves near the end of the run, reaching a maximum of 80% shots blocked. However, there is high variability in the performance. Finally, when the ball can come from any direction and position with variable velocity we see improvement in the agent, but not as high as the simpler scenarios.

In future work, we would like to try larger networks with bigger GPUs for training. It would also be worth investigating alternative methods such as the NAF, which can potentially outperform DDPG in this case. Additionally, prioritized replay memory should be revisited as a potential improvement. Most importantly, we will be trying the learned policies on the real robot hardware and during real game play. We expect transferring the learned policies to the real robots will take significant effort.

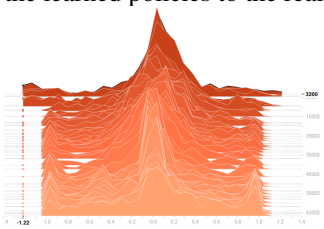


Figure 5: Histogram of actor action values during training.

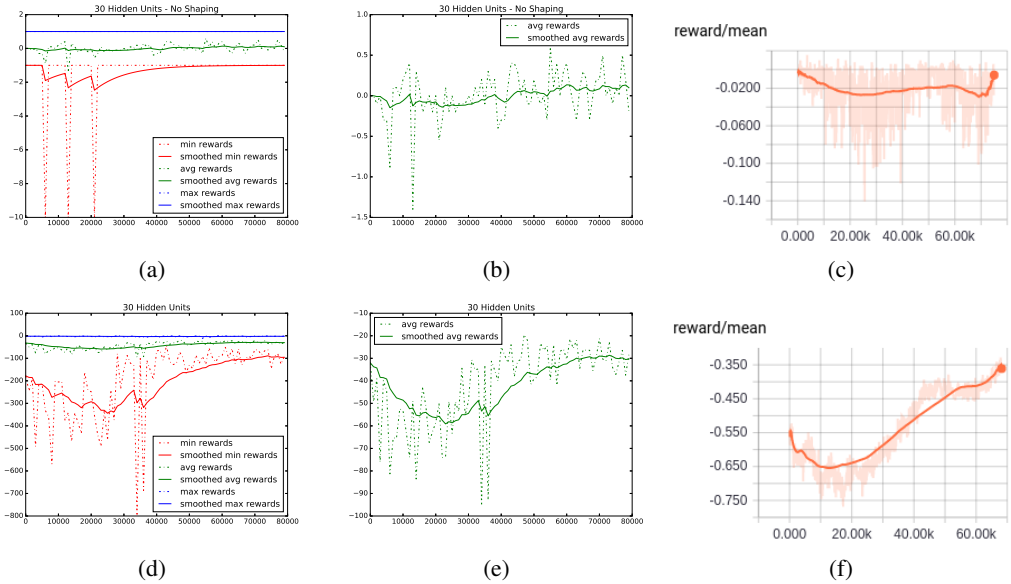


Figure 6: **Top row:** Rewards obtained using the unshaped reward function, and actor and critic networks with 3 layers of 30 hidden units each. 6a depicts the progress in terms of average, max and min reward obtained on 80k test episodes. 6b is a close-up on the average reward of the tests. 6c shows the average reward in each minibatch update during training. **Bottom row:** Rewards obtained using the shaped reward function with actor and critic networks of 3 layers and 30 hidden units each. 6d depicts the progress in terms of average, max and min reward obtained on 80k test episodes. 6e is a close-up on the average reward of the tests. 6f shows the average reward in each minibatch update during training. All figures were smoothed with an exponential weighted average, with factor 0.9.

References

[1] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dan Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. Software available from tensorflow.org. URL: `http://tensorflow.org/`.

[2] François Chollet. Keras. `https://github.com/fchollet/keras`, 2015.

[3] CMDragons. CMDragons, 2015. URL: `https://www.cs.cmu.edu/{~}robosoccer/small/`.

[4] The RoboCup Federation. RoboCup. URL: `http://www.robocup.org/`.

[5] Shixiang Gu, Timothy Lillicrap, Ilya Sutskever, and Sergey Levine. Continuous deep q-learning with model-based acceleration. 2016. URL: `http://arxiv.org/abs/1603.00748v1`, `arXiv:1603.00748`.

[6] Matthew Hausknecht, Yilun Chen, and Peter Stone. Deep imitation learning for parameterized action spaces. In *AAMAS Adaptive Learning Agents (ALA) Workshop*, Singapore, May 2016. URL: `http://www.cs.utexas.edu/users/ai-lab/?hausknecht2:aamasws16`.

[7] Matthew Hausknecht and Peter Stone. Deep reinforcement learning in parameterized action space. 2015. URL: `http://arxiv.org/abs/1511.04143v4`, `arXiv:1511.04143`.

[8] Shivaram Kalyanakrishnan, Yaxin Liu, and Peter Stone. Half field offense in RoboCup soccer: A multiagent reinforcement learning case study. In Gerhard Lakemeyer, Elizabeth Sklar, Domenico Sorenti, and Tomoichi Takahashi, editors, *RoboCup-2006: Robot Soccer World Cup X*, volume 4434 of *Lecture Notes in Artificial Intelligence*, pages 72–85. Springer Verlag, Berlin, 2007.

[9] Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[10] Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. 2015. URL: `http://arxiv.org/abs/1509.02971v5`, `arXiv:1509.02971`.

[11] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning. 2013. URL: `http://arxiv.org/abs/1312.5602v1`, `arXiv:1312.5602`.

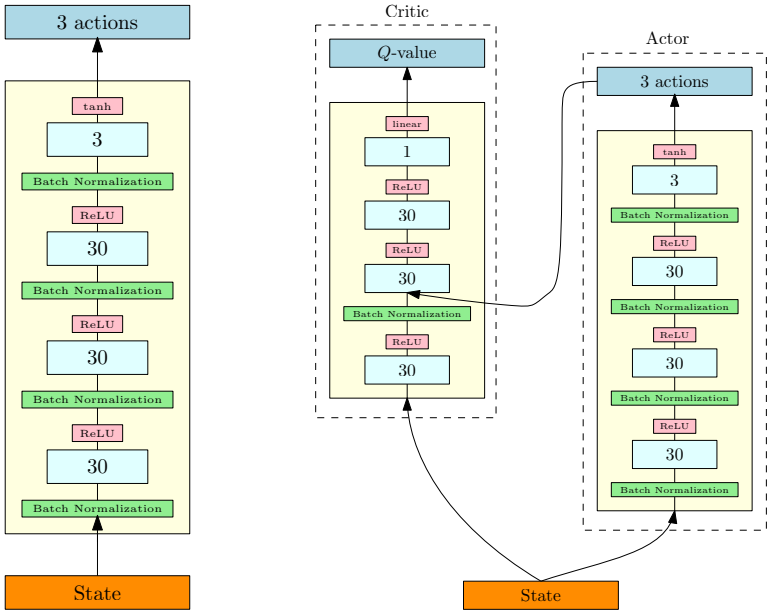
[12] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. URL: `http://dx.doi.org/10.1038/nature14236`, `doi:10.1038/nature14236`.

[13] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. 2015. URL: `http://arxiv.org/abs/1511.05952v4`, `arXiv:1511.05952`.

[14] David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016. URL: `http://dx.doi.org/10.1038/nature16961`, `doi:10.1038/nature16961`.

[15] Danny Zhu, Joydeep Biswas, and Manuela Veloso. *AutoRef: Towards Real-Robot Soccer Complete Automated Refereeing*, pages 419–430. RoboCup 2014: Robot World Cup XVIII. Springer Science + Business Media, 2015. URL: `http://dx.doi.org/10.1007/978-3-319-18615-3_34`, `doi:10.1007/978-3-319-18615-3_34`.

A Additional Figures



(a) DQN (b) DDPG

Figure 7: General architectures of the DQN and DDPG networks. Some experiments varied the number of layers and number of units in each layer. But all experiments used architectures similar to these.

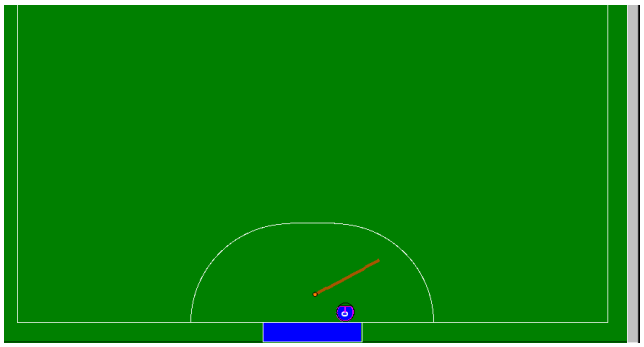


Figure 8: Screenshot of the our simulator showing the goalie blue circle, the spawned ball, orange streak and the goal, blue rectangle.